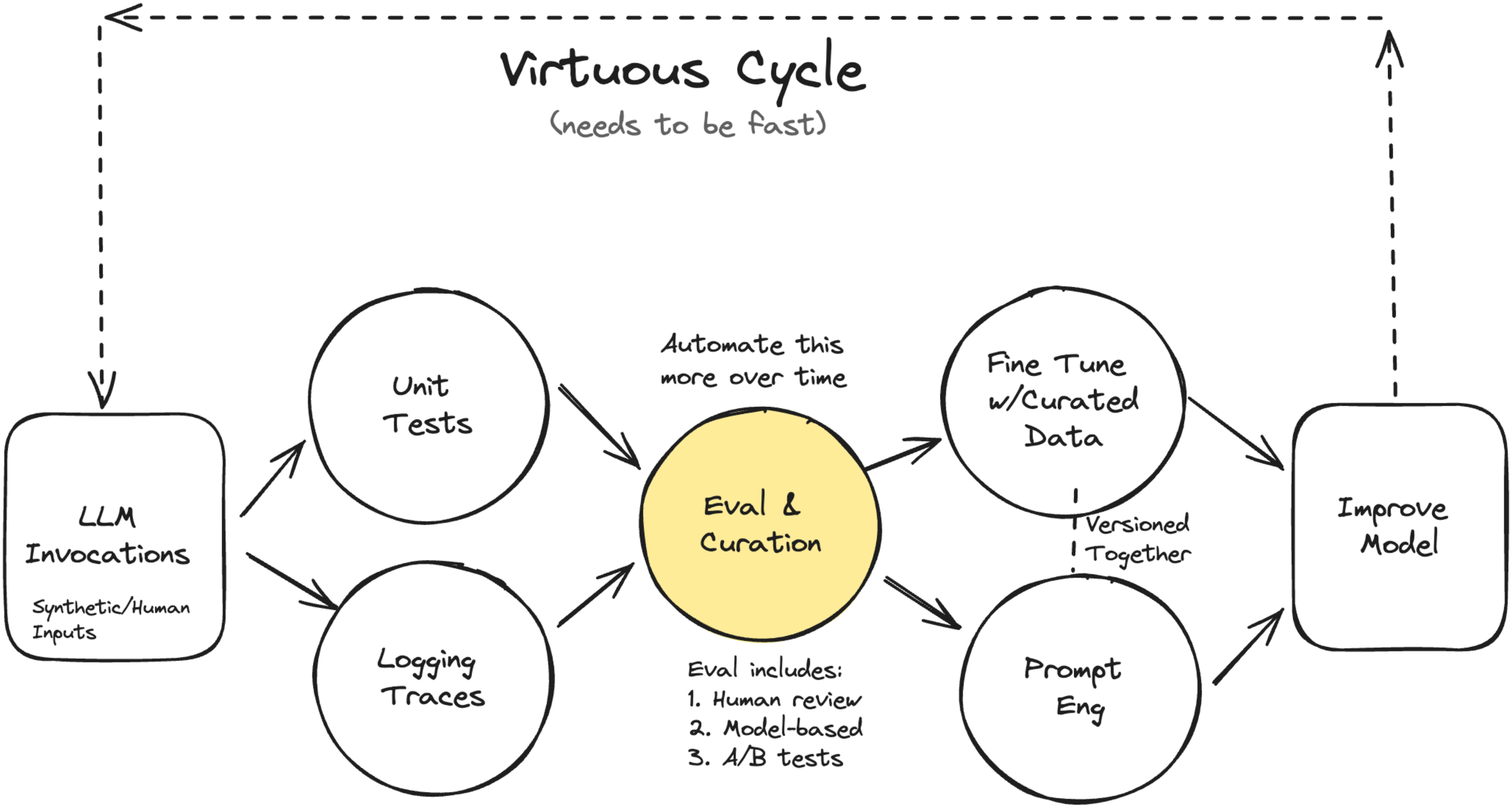
Week 5, part 1 of 2

Evals

"To know whether your AI product is working, you need a way to measure success beyond vibe checks."[1]



1. Hamel Husain, "Your AI Product Needs Evals", March 2024, "Motivation". https://hamel.dev/blog/posts/evals/ ↩


Virtuous Cycle
(needs to be fast)
LLM Invocations
Synthetic/Human Inputs
Unit Tests
Logging Traces
Automate this more over time
Eval & Curation
Eval includes:
1. Human review
2. Model-based
3. A/B tests
Fine Tune w/Curated Data
Versioned Together
Prompt Eng
Improve Model

Many failed ML/AI tools are only vibe checked

– Try a few prompts
– Get a few training scores
– Model(s) don't actually meet the requirements

Benchmarks are not your eval!!

"Once a dataset is published publicly in plain text, it will end up in model training datasets."[1]

1. Clementine Fourrier and the Hugging Face community, *The LLM Evaluation Guidebook*, "Automated benchmarks: basics". https://github.com/huggingface/evaluation-guidebook/blob/main/contents/automated-benchmarks/basics.md ↩

Three ways to evaluate a model[1]

- **Level 1, unit tests**
- **Level 2, human and model eval**
- **Level 3, A/B testing**

1. Hamel Husain, "Your AI Product Needs Evals", March 2024, "The Types Of Evaluation". https://hamel.dev/blog/posts/evals/ ↩

Unit Tests / Code Grading

Code grading pros

- Reproducible: a deterministic grader gives the same score for the same saved outputs.
- Cheap at scale: thousands of cases on every change.
- Exact: easy to identify failing cases.[1]

1. *The LLM Evaluation Guidebook*, "Automated benchmarks: basics", lists consistency, cost, and understandability as the advantages. https://github.com/huggingface/evaluation-guidebook/blob/main/contents/automated-benchmarks/basics.md ↩

Code grading cons

"More complex capabilities, on the other hand, are harder to decompose into well-defined and precise tasks."[1]

Is the summary faithful? Is the answer helpful?

1. *The LLM Evaluation Guidebook*, "Automated benchmarks: basics". https://github.com/huggingface/evaluation-guidebook/blob/main/contents/automated-benchmarks/basics.md ↩

START WITH ERROR ANALYSIS (IF YOU CAN)

Read your traces before you pick metrics.

"You must remove all friction from the process of looking at data."[1]

1. Hamel Husain, "Your AI Product Needs Evals", March 2024, "Looking at Your Traces". https://hamel.dev/blog/posts/evals/ ↩

Error analysis

1. Pull a batch of real traces.
2. For each failure, write one sentence: what went wrong.
3. Group the sentences and count.
4. Choose the first eval by severity as well as frequency.

The intern test

"If you took the exact input to the language model, including the context, and gave it to an average college student in the relevant major as a task, could they succeed?"[1]

1. Eugene Yan et al., "What We've Learned From A Year of Building with LLMs", June 2024, section 1.4.3. https://applied-llms.org/ ↩

ASSERTIONS FIRST

"Create unit tests (i.e., assertions) consisting of samples of inputs and outputs from production, with expectations for outputs based on at least three criteria."[1]

1. Eugene Yan et al., "What We've Learned From A Year of Building with LLMs", June 2024, section 1.4.1. https://applied-llms.org/ ↩

The flywheel

"Evaluation systems create a flywheel that allows you to iterate very quickly."[1]

1. Hamel Husain, "Your AI Product Needs Evals", March 2024, "Conclusion". https://hamel.dev/blog/posts/evals/ ↩

Demo: build an eval

One task, twenty traces, four assertions, one regression.

`demos/5.1-evals/build-an-eval.ipynb`

The task

Support email in, JSON ticket out:

- order_id, category, refund_requested, refund_amount, summary
- category is one of: refund, delivery, product, other

Assertions

1. The output parses as JSON.
2. The JSON matches the schema, category in the enum.
3. The order id appears verbatim in the email.
4. The refund amount is not negative.

Eval Figjam



password: `iluvcomp3110`

RECAP

- Vibe checks catch only the failures you thought of.
- Benchmarks rank models. Your product needs its own set.
- Error analysis picks the evals: read, label, group, count.
- Assertions grade what code can check.

Next hour: LLM as a judge, for everything code cannot check.